

人工智能数学原理与算法 B

第三次实验

逻辑回归与贷款违约预测

分阶段实验手册

主讲 钟志诚

整理 曾有成

单位 中国科学技术大学

人工智能与数据科学学院

从数据到可解释的分类流程

数据清洗与划分 → 数值表示 → 线性得分与概率 → 交叉熵与正则化 → 梯度更新 → 独立评估。

2026 年秋季 建议 3 学时 (135 分钟)

C0-C7 分阶段完成；日期与提交平台以课程通知为准。

随包数据为天池真实贷款记录的固定课堂子集。

目录

1 如何使用这份实验材料	1
2 135 分钟课堂路线	2
3 C0–C2：从表格到张量	3
4 C3–C4：批次与模型	4
5 C5–C6：损失、正则化与更新	5
6 C7：选方案，再做最终评价	6
7 实验报告与提交	7
8 提交前检查、评分与拓展	9
9 排错、结果解读与资料对照	10

1 如何使用这份实验材料

1.1 课前准备与文件入口

先阅读《实验 03_ 前置讲义》的第 1–6 节，课前约 30 分钟；课堂通过手算与代码继续学习第 7–10 节。实验总时长建议 135 分钟，不把课前安装时间算入课堂。

沿用实验 01 的课程环境。终端进入本实验的 `code/` 目录，运行：

```
conda activate ustc-ai-experiments-26fall-B
python -c "import torch, pandas, matplotlib; print(torch.__version__)"
python checkpoints.py C0
```

若缺少模块，在已激活环境运行 `python -m pip install -r requirements.txt`。本实验在 CPU（Central Processing Unit，中央处理器）上运行即可，无需配置加速硬件。依赖文件列出本次验证过的版本，已有可用环境不必重复安装。

文件	用途
<code>student.py</code>	唯一主要补全文件，所有 TODO 均有 Hint
<code>checkpoints.py</code>	C0–C7 小样例检查；只检查指定阶段
<code>support.py</code> <code>workflow.py</code>	已提供的读表、日期编码、绘图、模型选择与测试流程
<code>run_lab.py</code>	补全后运行整个实验的普通 Python 入口
<code>logic.ipynb</code>	按 checkpoint 组织的 Notebook 入口，复用同一份学生代码
<code>data/classroom/</code>	真实贷款课堂子集与划分记录，可离线运行
<code>report-template.md</code>	实验记录模板，含每阶段证据与总结

1.2 每一阶段都按同一节奏进行

阅读目标，先预测小例子的输出，再运行并改动一个值；补全指定函数；运行对应检查；把实现思路和一条数值验证留作报告素材，按第 7 节整理。Ck PASS 只表示所列小样例通过，不能代替解释与完整实验。

Notebook 通过普通 `import student` 调用 `student.py`。修改该文件后，重启内核并从头执行到当前阶段，避免用到旧函数；不要在两处维护不同答案。初始骨架中的 `return None` 是明确的待完成占位，不是算法返回值。

不要跳过形状与数据来源

随包数据不是教材原始贷款数据。先完成 C0；C1–C7 未完成时出现断言失败或类型报错是预期行为，按 checkpoint 提示定位。不要把全部报错都归因于环境。

2 135 分钟课堂路线

2.1 教师活动、学生活动与可观察产出

阶段/分钟	内容	教师活动	学生活动与产出
C0 / 10	环境、形状	连接前两次实验，演示矩阵乘法	运行检查，记录得分与形状
C1 / 15	读表、编码	说明标签和编号职责	补两个函数，列出缺失数和编码结果
C2 / 20	填充、尺度	手算均值与训练统计量	补三个函数，输出张量维度
C3 / 15	批次	用五条数据演示成对取样	补数据集方法和 loader，观察末批
C4 / 20	得分与概率	画出 sigmoid，解释参数对象	补模型，手算与代码核对
C5 / 20	损失、正则	推导交叉熵，讲解接口	补损失和惩罚，核对数值
C6 / 20	一次更新	板书梯度与五步训练循环	补训练，验证两次连续更新
C7 / 15	选择、评价	讲解四类计数和三种数据集	补指标，运行方案比较与退出卡

合计 $10 + 15 + 20 + 15 + 20 + 20 + 20 + 15 = 135$ 分钟。每段讲解后立即上机，C6/C7 完整记录可课后完善；课堂不临时增加必须完成的大范围调参。

2.2 完成标准与依赖关系

C0 独立可运行。C1、C2 的小测试彼此独立；C3 使用提供的小张量；C4 建立模型；C5 需要 C4；C6 需要 C3–C5；C7 指标小测试可独立完成，但完整实验需要全部阶段。

必做：C0–C7、课堂手算、默认三方案比较、冻结后最终测试、错误分析与实验反馈。**选做：**在看测试结果之前，只在训练/验证上改变学习率或选择阈值。若测试已揭晓，再修改方案仅算探索，不能把同一测试集当成新的独立证据。

建议板书

左侧：原始表、训练统计量和张量形状；中间： $\mathbf{X}\boldsymbol{\theta} \rightarrow \mathbf{p} \rightarrow R \rightarrow \nabla R$ ；右侧：训练/验证/测试职责与混淆矩阵。各记号定义见前置讲义第 3、5–9 节。

检查规则

每个 checkpoint 在课堂自查时核对一条数值输出，并说清自己的解释。报告逐项记录关键实现、小例子的输入及预期与实际输出、结果解释；综合训练图表集中展示。未通过时记录具体函数、现象与当前理解。

3 C0–C2: 从表格到张量

3.1 C0: 先运行一个已完成的小例子

运行 `python checkpoints.py C0`, 预期显示课堂训练表 (8000,47)、缺失标签 0、矩阵乘法结果 $\begin{bmatrix} 2.0 \\ -1.0 \end{bmatrix}$ 。原表保留 47 列, 本次只选八个输入字段, 编号与标签不进入模型。把例子中的权重 1 改为 2, 先预测输出再核对; 改回后继续。

3.2 C1: 缺失标签与就业年限

在 `student.py` 补 `filter_labels` 和 `encode_employment`。前者只删标签缺失行, 不在这一步填充输入; 后者处理缺失值、单数 `year`、复数 `years` 和两个特殊档位。

```
import pandas as pd
import support
train = support.load_split(support.DATA_DIR, "train")
print(train.head())
print(train.isna().sum())
print(train["isDefault"].value_counts(dropna=False))
```

运行 `python checkpoints.py C1`。应保留示例中特征缺失但标签有效的行; 年限编码依次为 1、2、4、12。用自己的话说明为什么缺失标签不能填均值。

3.3 C2: 只拟合训练统计量

补 `fit_statistics`、`apply_statistics` 和 `to_tensors`。前者返回训练均值和填补后的总体标准差; 第二个只应用已有规则; 第三个转换为浮点张量, 首列加 1, 标签变为列向量。

运行 `python checkpoints.py C2`。小样例均值为 2, 尺度约 0.816497。完成 C1/C2 后运行:

```
import student
train, val, state = support.prepare_train_val(
    student, support.DATA_DIR)
print(train[0].shape, train[1].shape)
print(val[0].shape, val[1].shape)
```

预期训练为 (8000,9) 与 (8000,1), 验证为 (1000,9) 与 (1000,1)。训练与验证均无缺失标签; 训练就业年限缺失 503 条、债务收入比缺失 1 条, 验证就业年限缺失 48 条; 测试文件尚未读取。C1 小例子专门练习删除缺失标签。

验收证据

列出八列的顺序; 解释新增常数列为何放在首列; 用数值证明验证值 100 没有参与计算填充值。不要把标签、编号或原始行索引加入特征矩阵。

4 C3–C4: 批次与模型

4.1 C3: 构造 Dataset 和 DataLoader

补 `LoanDataset.__len__`、`__getitem__` 和 `make_loader`。构造函数已经保存特征与标签，不必另做数据复制。

```
dataset = student.LoanDataset(*train)
loader = student.make_loader(dataset, 64, True)
print(len(dataset))
for X_batch, y_batch in loader:
    print(X_batch.shape, y_batch.shape)
```

星号 `*train` 把二元组展开为两个参数，等价于传入 `train[0]`，`train[1]`。代码应输出 8000 个样本，125 批且每批 64 条。小自检命令为 `python checkpoints.py C3`，其五条数据应分为 2、2、1。

4.2 C4: 注册参数，计算得分

补 `sigmoid`、`LogisticRegression.__init__` 和 `forward`。`nn.Module` 是可训练模型的基类，`nn.Parameter` 用来注册权重，使优化器能找到它。输入已经带常数列，因此这里不再增加单独偏置。

```
model = student.LogisticRegression(train[0].shape[1])
logits = model(train[0][:3])
probabilities = student.sigmoid(logits)
print(logits.shape, probabilities.shape)
print(probabilities)
```

零初始化时前三条概率都是 0.5；单层逻辑回归不存在隐藏单元的对称性问题，可以这样初始化。

先手算，再检查

三个参数为 $(1, 2, -1)^T$ ，输入为 $(1, 3, 2)^T$ ，得分为 5、概率约为 0.993307。这里 T 表示转置，括号内是列向量；本例仅有两个非恒定特征，与完整数据的八维输入分开使用。

运行 `python checkpoints.py C4`。若返回 0.993307 导致得分测试失败，检查是不是在 `forward` 中过早调用了 `sigmoid`。若形状是 $(B, 9)$ ，检查是否把矩阵乘法 `@` 写成逐元素乘法 `*`。

验收证据

记录首批与末批形状，解释 `shuffle` 为什么只用于训练；画出“输入矩阵 $\rightarrow$ 得分列 $\rightarrow$ 概率列”的形状链。

5 C5–C6: 损失、正则化与更新

5.1 C5: 确认损失输入与惩罚范围

补 `data_loss` 和模型的 `penalty`。BCE (Binary Cross-Entropy, 二元交叉熵) 在本程序通过 `BCEWithLogitsLoss` 计算, 输入为得分, 不是概率。`penalty` 只取 `theta[1:]`, 不含首项截距。

L1 (L1 Norm, 一范数) 惩罚是权重绝对值之和; L2 (L2 Norm, 二范数) 惩罚在本实验明确约定为权重平方和的一半。`kind="none"` 返回同设备的标量零, 训练总目标等于数据损失。

运行 `python checkpoints.py C5`。得分全为 0 时, 平均损失约 0.693147; 参数 $(10, 3, -4)^T$ 的 L1 与 L2 惩罚为 7、12.5。自检还检查极端得分的有限损失与惩罚梯度, 防止悄悄重复 sigmoid 或误惩罚截距。

5.2 C6: 补全一个可解释的训练循环

补 `train_epoch`。训练器使用 SGD (Stochastic Gradient Descent, 随机梯度下降); 本实验每次基于一批样本更新。按以下顺序连通计算:

1. `optimizer.zero_grad()`: 清除旧梯度。
2. 用 `model(X)` 得到得分, 计算平均数据损失。
3. 将 `strength * model.penalty(kind)` 加到数据损失上。
4. 对总目标调用 `backward()`, 再调用 `optimizer.step()`。
5. 仅在记录时提取普通数值; 按每批实际样本数累加数据损失。

运行 `python checkpoints.py C6`。两条手算样本、零初始化、学习率 0.4、无正则化时, 更新后参数应为 $(0, 0.1, -0.1)^T$ 。它还核验第二步更新, 检查是否忘了清梯度。

记录和优化不要混为一谈

`train_epoch` 返回各批更新前数据损失的加权平均, 用于理解迭代过程; 正式损失曲线由 `workflow.py` 在每轮结束后, 固定同一组参数重新评价整个训练与验证集, 二者定义不同。

验收证据

手写两条样本的梯度; 保存第一次更新参数; 用五条样本、批量 2 的小例子, 解释末批只有 1 条时为何不能把各批平均损失直接等权平均。

6 C7：选方案，再做最终评价

6.1 完成指标，运行验证实验

补 `binary_metrics`，预测概率大于或等于阈值时判为违约。先完成 C7 的小样例检查，再运行全部自检与候选方案训练：

```
python checkpoints.py all
python run_lab.py
```

默认比较无正则、L2 系数 0.01、L1 系数 0.01 三组。每组学习率 0.05、最多 60 轮、批量大小 64、种子 42。每次候选训练前重设种子，零初始化权重；比较条件保持一致。

每组保存验证数据损失最小的那一轮，再在三组中选择验证数据损失最小者；保存的是参数副本而非仍随训练变化的引用。固定阈值 0.5。`outputs-tianchi-balanced/` 中会出现：

- `candidates.csv`：三组的最优轮数、验证损失和准确率。
- `none_loss.png`、`l1_loss.png`、`l2_loss.png`：无正则数据损失曲线；同名 `history` 表保存数值。
- `selection.json`、`preprocessing.json`、`selected.pt`：冻结的模型设置、预处理统计量与权重。
- `validation_thresholds.csv`：验证集上三种阈值的指标和教学代价；`example_cost` 按“漏判代价 2、误报代价 1”计算，仅用于讨论。

6.2 冻结后，单独运行最终测试

检查上述文件，写下选用哪组、哪轮及理由，然后执行：

```
python run_lab.py --final-test
```

这个命令读取已经保存的权重和预处理规则，才首次读取测试文件，输出 `test_metrics.json` 与 `test_predictions.csv`。请沿用同一数据目录和输出目录。完成后不再基于测试结果重调参数。

TP (True Positive, 真正例)、FP (False Positive, 假正例)、FN (False Negative, 假负例)、TN (True Negative, 真负例) 均以“违约”为正类。输出包含准确率、精确率、召回率和 F1 分数 (F1 Score)，并给出常数基线准确率；训练两类相同时，预先约定总预测未违约。分母为 0 的指标约定记为 0。

验收证据

展示一张曲线、一张混淆矩阵；说明多数类基线和模型的差别。若准确率尚可但漏判较多，指出应查看的指标，并在验证阈值表中解释误报与漏判如何变化。

7 实验报告与提交

7.1 四部分报告结构

提交 `report.pdf`。PDF (Portable Document Format, 可移植文档格式) 报告首页写实验名称、学号和姓名; 可填写 `report-template.md` 后导出, 无需同时提交模板原文件。沿用实验 01 的提交方式, 在报告中增加分阶段实现与验证。

1. **实验概况与完成总览**。简述任务, 列出 C0-C7 的完成状态; 未完成项写清函数和具体问题。
2. **C0-C7 实现与验证**。按下页要求逐项写关键实现、小例子的输入及预期与实际输出、结果解释。C0 记录提供的示例及修改观察; C6 保留一次完整参数更新的简短手算与代码对照。
3. **综合实验结果**。说明数据来源、清洗前后三份样本数和训练统计量处理; 列出种子、学习率、批量大小、最大轮数。比较三方案的系数、最优轮数、验证数据损失和准确率; 测试前写下选择及理由。插入选定方案的一张训练/验证损失曲线。列出测试准确率、精确率、召回率、F1 分数 (F1 Score)、多数类基线和混淆矩阵; 矩阵行是真实类别、列是预测类别, 0 为未违约、1 为违约。
4. **结果分析与总结**。解释损失变化、误报与漏判、多数类基线; 引用验证阈值表中至少两个阈值, 讨论任务代价。不根据测试结果重新调参; 说明一项局限及实验收获或建议。

把代码与证据对应起来

关键代码通常摘录约 3-10 行, 注明文件与函数; 以表达完整为准, 不贴整个文件。预期值来自手算或手册, 实际值必须自己运行得到; 只贴 **PASS** 不能说明实现。综合图表集中放在报告第 3 部分, 各检查点可直接引用。未完成项如实说明, 不按篇幅或截图数量评价。

报告中的图表与数值必须来自本次提交代码, 不得把本次抽样称为教材原有划分或照抄教师结果。无需安装证明、完整日志或整篇理论推导。

7.2 压缩包保留哪些文件

提交 学号_姓名_lab03.zip, 解压后根目录包含 `report.pdf` 与 `code/`。后者保留 `student.py`、`run_lab.py`、`support.py`、`workflow.py`、`checkpoints.py`、`requirements.txt`, 以及 `data/classroom/` 中三份数据、划分清单与数据说明; 不打包天池原始大文件。

采用 Notebook 入口时另附 `logic.ipynb`, 仍须保留学生脚本和辅助模块。与实验 01 不同, 当前训练入口会调用检查模块, 所以 `checkpoints.py` 也须随包提交。完整目录示意见本实验 `README.md`。

输出目录中的原始表、权重与预测明细自行保留即可; 无需重复提交报告中已有的图表。不打包 Python 环境、安装包、缓存、教师答案与课程讲义。截止时间和提交平台以课程通知为准。

7.3 每个检查点在报告中写什么

阶段	实现定位或观察对象	至少记录的一项数值验证及解释
C0	提供的矩阵乘法示例；运行入口与版本	输入、权重与输出形状；改动一个权重前后的预期及实际得分。说明变化原因。
C1	<code>filter_labels</code> 、 <code>encode_employment</code>	含缺失标签的小表保留多少行；特殊年限、普通年限与缺失值的编码。解释标签与输入的区别。
C2	<code>fit_statistics</code> 、 <code>apply_statistics</code> 、 <code>to_tensors</code>	小例子的均值、填补后标准差、验证转换值与张量形状/类型；改变验证数据后训练统计量不变。
C3	<code>LoanDataset</code> 的两个方法、 <code>make_loader</code>	同索引的特征和标签；样本数不能整除批量大小时的各批大小与总数。说明打乱规则。
C4	<code>sigmoid</code> 、模型初始化与 <code>forward</code>	一组输入和参数对应的得分、概率与形状。解释参数注册，以及得分和概率的区别。
C5	<code>data_loss</code> 、 <code>penalty</code>	零得分损失；一组参数的三种惩罚。只改截距后惩罚不变；区分数据损失与总目标。
C6	<code>train_epoch</code> 的五步更新	两样本例子的初始参数、概率、梯度、学习率及一次更新的手算/运行对照；第二次更新结果。解释清梯度与按样本数加权。
C7	<code>binary_metrics</code> 、提供的选择与测试流程	小例子在两个阈值下的类别、混淆矩阵和指标，含概率等于阈值的情况。说明验证选择与冻结测试的职责。

一条记录应能被复核

例如 C2：写出自己选取的训练列与验证值，摘录拟合和复用统计量的代码；列出预期与实际均值、尺度和转换值，最后解释验证值为何不能参与拟合。不要只写“标准化完成”或“C2 通过”。具体函数和填写位置见报告模板第 2 节。

8 提交前检查、评分与拓展

8.1 检查可复现性，再打包

1. 运行 `python checkpoints.py all` 核对实现，不要求验证截图。
2. 确认报告来自待提交代码的训练、验证选择与冻结后最终测试；测试结果揭晓后不再据此调参。
3. 打开 `report.pdf` 检查文字、曲线与数值。解压到另一目录，运行 C0 和全部自检，确认代码与数据齐全；不必为打包重复测试选模。

公开检查帮助定位问题，不等于全部评分规则。尚未解决的问题在报告中如实注明，保留已完成实现。

8.2 评分结构对齐

比例	验收内容	对应阶段
30%	环境 5%、读表 5%、缺失值与训练统计量 10%、数据加载 10%	C0–C3
40%	参数初始化 10%、sigmoid 10%、前向传播 10%、正则化 10%	C4–C5
30%	损失 10%、训练及冻结后的最终评价 10%、收获与建议 10%	C5–C7

此建议沿用指定指导书第 1.6 节的 30/40/30 结构，按 checkpoint 核验实现与防泄漏要求。没有“必须达到某个准确率”的硬门槛。PASS 不替代理解；能说明尚未解决的问题也是有效实验记录。

8.3 课后问题与参考要点

1. 缺失标签能否按多数类补齐？不能，伪造的监督信号会改变训练与评估含义。
2. 预测全为未违约为什么可能准确率很高？类别不平衡；同时看正类召回率与多数类基线。
3. 过大正则化一定改善泛化吗？不会；它可能抑制必要的信号，须用验证集比较。
4. 为什么本实验不要求手写反向传播？先建立损失、计算图与更新的对应；后续实验再实现和比较梯度算法。

8.4 课堂数据与自备划分

默认从 `code/data/classroom/` 读取真实贷款子集，结果写入 `code/outputs-tianchi-balanced/`，不能复用旧模拟权重。天池 `testA.csv` 没有标签，提交示例中的 0.5 也不是真值；课堂三份数据均从带标签训练文件中重新划分。

自备划分目录须含 `train.csv`、`val.csv`、`test.csv`，满足八字段与真实标签约定。`dti` 保留原尺度，不直接替换成模拟负债比例。更多字段不会自动进入模型，须核对类型、单位与预测时可用性后再扩展。

```
python run_lab.py --data-dir ../course-data --output outputs-course
python run_lab.py --data-dir ../course-data --output outputs-course --final-
test
```

9 排错、结果解读与资料对照

9.1 常见问题按现象定位

现象	首先检查
找不到 <code>torch</code>	终端解释器是否为课程环境；Notebook 内核是否一致
<code>NoneType</code> 报错	当前 checkpoint 的占位返回是否已替换，是否漏补依赖阶段
输入/标签形状不匹配	标签是否为 <code>(B, 1)</code> ；是否少了增广常数列
所有概率都相同	参数是否注册为 <code>nn.Parameter</code> ；训练是否执行更新
损失不降或出现非数值	特征是否填充且标准化；学习率是否过大；损失是否接收原始得分
第二步手算不一致	是否每批清梯度；是否对总目标而非记录值反向传播
预测与标签错位	是否独立打乱了标签；Dataset 是否按相同索引返回二者
验证很高、测试很差	是否混用数据、把标签放入特征、根据测试结果选参数

9.2 参考运行仅用于核对流程

本次真实贷款子集使用 8000/1000/1000 条记录；按指导书将训练正负各设为 4000 条，验证与测试各含 200 条违约。零初始化、训练种子 42、学习率 0.05、批量 64，每组最多 60 轮。按验证数据损失选择 L1 系数 0.01 的第 3 轮，阈值固定为 0.5。

验证损失约 0.648418、准确率 0.644、召回率 0.65；测试损失约 0.620095、准确率 0.667、精确率约 0.320、召回率 0.59，混淆矩阵为真负例 549、假正例 251、假负例 82、真正例 118。常数基线准确率为 0.8。

原自然比例训练的测试召回率为 6%，平衡后为 59%，但误报由 3 增至 251。**这是同一已查看测试集上的教学对照，不是新的盲测，也不是教材原结果。**验证阈值 0.3/0.5/0.7 时召回率为 95%/65%/19%，示例代价为 699/426/373；不能仅凭召回率判断优劣。平衡训练改变类别先验，输出未经概率校准。不同库版本可能影响结果，不按逐位相等评分。

9.3 资料与改编对应

指定《人工智能数学原理与算法：实验指导书（最终）.pdf》位于项目上级目录。第 1.4 节对应 C1–C2；第 1.5.2 节对应 C3–C6；第 1.5.3 节与第 1.6 节对应 C7、提交及评分建议。页码为书内第 1–13 页，文件第 15–27 页。

仓库 `source/experiment/Logic.ipynb`、`source/experiment/utils.py` 用于核对旧版入口与字段转换；前置讲义列出理论课件与课程总表路径。随包 `code/data/README.md` 与 `teacher/prepare_classroom_data.py` 给出真实数据来源及抽样规则；教师实现与验证说明位于 `teacher/`，学生下载包不包含该目录。报告提交方式参考同级实验 01 的实验手册第 7 节及报告模板；实验 02 不另交报告。分阶段记录方式借鉴用户提供的《数据科学基础》个人 HW1 成稿第 5、9–10 页的“任务、代码、输出、结论”组织；只借鉴结构，外部资料位置见教师说明。

相较原版，明确得分/概率接口，保存训练统计量，处理 **1 year** 单数形式，并用参数首项表示截距；零初始化和三组预设比较是课堂改编。数学符号完整定义及分类索引见《前置讲义》第 10 节。